Specialization & Generalization

A practical guide to superclasses, subclasses, inheritance, constraints, relational mapping and viva answers.

Implementation update: The Next.js/SQLite website now has an additive Person supertype and Student/Faculty subtype links, plus a weak contact entity with a partial key. The original student and faculty tables and account roles remain unchanged. The Person links are partial and overlapping and must be created explicitly in the admin `/eer` screen. The independent PostgreSQL/FastAPI build is unchanged.

1. Specialization: top-down

Specialization divides a broad entity type into more specific subtypes based on distinguishing properties or responsibilities. The broad type is the **superclass/supertype**; the specific types are **subclasses/subtypes**.

```
Person
 ├── Student  (enrollment number, admission year)
 └── Faculty  (employee code, designation)
```

A Student **is a** Person, and a Faculty member **is a** Person. Shared attributes such as name, email and date of birth belong to Person; subtype-specific attributes belong to the subtype.

Why use it?

- Represent meaningful categories while storing common facts in a consistent place.
- Apply shared relationships or constraints to the superclass.
- Keep subtype-specific properties out of unrelated records.

> Viva: “Specialization is top-down: start with Person, then identify Student and Faculty as more specific entity types. A subtype inherits its identity and common attributes from the supertype.”

Not every category needs specialization. A status column such as Active/Inactive describes state, not necessarily a different entity subtype.

2. Generalization: bottom-up

Generalization starts with multiple entity types and recognizes a shared higher-level abstraction. Starting with Student and Faculty, we notice shared names/contact information and propose Person as their superclass.

Concept	Direction	Question asked
Specialization	General → specific	Which distinct subtypes of Person are needed?
Generalization	Specific → general	Which common entity can unify Student and Faculty?

The final hierarchy can look identical; the distinction is the design reasoning used to reach it.

Inheritance and ISA

Subtypes inherit the supertype identity, attributes and applicable relationship participation. A subtype row represents the same real-world entity, not an independent copy. In a shared-primary-key relational design, student.person_id is both its PK and an FK to person.person_id.

Specialization versus aggregation and ordinary relationships

- **ISA:** Student is a Person — subtype membership.
- **Association:** Student enrolls in Offering — two different entities linked through enrollment, not inheritance.
- **Part-of:** a component belongs to a whole; it is not automatically an ISA relation.
- **EER aggregation:** treating a relationship and its participating entities as a higher-level unit for another relationship.
- **SQL aggregation:** COUNT, SUM and GROUP BY summarize rows; this is a different meaning of “aggregation.”

> Viva: “Generalization is bottom-up. It extracts common features from existing entity types into a supertype. It is not a SQL GROUP BY operation and it is not the same as a foreign-key relationship.”

3. The two independent constraints

A. Disjoint versus overlapping

Disjoint: one superclass entity may belong to at most one subtype in this specialization. For example, an account with exactly one stored admin/student role behaves as an exclusive role choice, but that alone does not create subtype tables.

Overlapping: a person may belong to multiple subtypes, such as a postgraduate student who is also employed as faculty. The institution's business rules determine whether this is permitted.

B. Total versus partial completeness

Total: every superclass entity must belong to at least one subtype. For example, if Person exists only for Student or Faculty records, no unrelated Person rows should exist.

Partial: a superclass entity can exist without belonging to the modeled subtypes, such as a visitor or external collaborator in a broader Person registry.

Combination	Meaning
Total + disjoint	Every Person is exactly one modeled subtype.
Total + overlapping	Every Person is at least one subtype and may be several.
Partial + disjoint	A Person may be none or one subtype.
Partial + overlapping	A Person may be none, one or several subtypes.

How membership is determined

Predicate/attribute-defined: membership follows a condition or discriminator value. **User-defined:** membership is explicitly assigned according to application rules. For overlapping categories, a single exclusive discriminator may be insufficient.

Typical EER notation: ISA connectors or a specialization circle; d for disjoint and o for overlapping; double line for total participation, single for partial. Notation conventions vary by textbook. No diagram is required to explain these definitions.

4. Mapping a hierarchy into SQL tables

Strategy	Benefits	Trade-offs
Superclass + subtype tables	Common facts stored once; subtype fields isolated; shared identity explicit.	Joins needed. Disjointness/completeness may need extra constraints or application logic.
Single table + discriminator	Simple shared queries; one identity table; no join for common/subtype attributes.	Many nullable subtype columns; conditional validation; overlapping categories are less natural.
Concrete subtype tables only	Simple independent subtype access, no superclass join.	Common attributes repeat in schema; cross-type identity and overlapping membership are harder to maintain.

Proposed shared-primary-key example (not current CampusOS schema)

```
CREATE TABLE person (
  person_id INTEGER PRIMARY KEY,
  full_name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE
);
CREATE TABLE student_person (
  person_id INTEGER PRIMARY KEY
    REFERENCES person(person_id),
  enrollment_number TEXT NOT NULL UNIQUE
);
CREATE TABLE faculty_person (
  person_id INTEGER PRIMARY KEY
    REFERENCES person(person_id),
  employee_code TEXT NOT NULL UNIQUE
);
```

A subtype row must reference an existing Person. Its PK prevents duplicate membership within that subtype. The same person_id can appear in both subtype tables, so this base structure permits **overlap**. A Person with no subtype is allowed, so it is **partial** unless further rules are added.

> A foreign key alone does not enforce total specialization or disjointness. Enforcing those across tables requires deliberate transaction design, appropriate triggers/constraints or a schema strategy that makes invalid combinations impossible. SQLite does not offer PostgreSQL-style deferred constraint triggers.

5. What CampusOS actually implements

Actual table	Primary key	Meaning
student	student_id	Independent student entity with its own identity, contact and academic attributes.
faculty	faculty_id	Independent faculty entity with employee, contact and department attributes.
portal_account	account_id	Authentication identity, role, optional linked student_id and password hash/salt.
portal_session	token_hash	Expiring login session referencing account_id.

CampusOS has now added an optional `campus_person` identity table. The existing `student` and `faculty` tables remain their own legacy records and primary keys; `person_student` and `person_faculty` optionally link them. There is no automatic identity matching. Thus an unlinked student/faculty is valid; one Person may link both, only one, or neither subtype.

Account role rules

- role is restricted to admin or student.
- An admin account must have student_id NULL.
- A student account must have a non-NULL student_id referencing student.
- student_id is unique in portal_account, preventing multiple linked accounts for the same student under this design.
- The UI and server enforce role-specific access. This is **RBAC-like authorization with a single role field**, not Person–Student–Faculty subtype inheritance.

> “Our Next.js/SQLite build has an optional Person registry with partial, overlapping Student/Faculty subtype links. Administrators link legacy records only when they can verify identity. The original primary keys and account authorization model remain unchanged.”

The added `person_contact` table is a weak entity owned by `campus_person`. Its full PK is `(person_id, contact_type, contact_value)`; `(contact_type, contact_value)` is its owner-relative partial key. Multiple phone/email contacts can be recorded per Person. It does not replace the single-value student.phone column. Manage this additive feature at `/eer` in the SQLite website.

This identity/contact extension applies to the Next.js/SQLite build; it is not part of the original independent PostgreSQL/FastAPI schema.

6. Future extension and viva answers

How to introduce Person safely

1. Decide identity matching rules; do not merge people by name alone.
2. Choose overlap: can one person be both student and faculty? Choose total/partial completeness based on requirements.
3. Create Person and assign stable person IDs. Migrate verified common facts; resolve conflicting email/contact data.
4. Link student/faculty to Person while preserving existing IDs or deliberately migrate all references in a transaction-controlled process.
5. Adapt APIs, joins, forms and accounts. Decide whether accounts link to Person or a particular role identity.
6. Test that enrollments, grades, attendance, assignments and portal authorization remain unchanged.

Question	Ready answer
Specialization vs generalization?	Top-down subtype discovery versus bottom-up common-supertype discovery.
What does a subtype inherit?	Supertype identity, common attributes and applicable relationships.
Is a role column specialization?	It can help implement a conceptual single-table hierarchy, but by itself it does not prove one. CampusOS uses it for authorization.
Can one person be student and faculty?	An overlapping Person hierarchy could represent this. Current CampusOS stores independent entities and does not unify their identity.
Do foreign keys enforce total membership?	No. Subtype → Person FK requires a parent, not that every Person has a subtype.
Is Person implemented here?	No. It is a proposed example, not the deployed schema.
Is enrollment a subtype?	No. It is an associative entity linking student and course offering.

> 30-second summary: “Specialization starts with a general entity and creates specific subtypes; generalization works in the opposite direction. Subtypes inherit common identity and attributes. Disjointness controls whether multiple membership is allowed; completeness controls whether every supertype must have a subtype. CampusOS currently has independent Student and Faculty tables and role-based accounts, not a Person ISA hierarchy.”

Source mapping: web/lib/schema.mjs and web/lib/portal.mjs.
Editable source: learnit/Specialization_Generalization.html